Parallel Programming · Winter term 2026/27

Exercise 3



Task 1 · `Speedup Curves`

Create speedup curves using *LibreOffice Calc* or *MS Excel*. Other tools work just as well, for example Python with *matplotlib* or *gnuplot*.

- Use the **y-axis** for the speedup (apply a **logarithmic scale**).
- Use the **x-axis** for the number of cores (1–128).
- Plot the theoretical speedup according to **Amdahl's Law** for the following sequential fractions:

$$f = 0.2,\ 0.1,\ 0.05,\ 0.01,\ 0.001$$

- Additionally, include an ideal (perfectly scalable) curve.

Task 2 · Efficiency, Overhead, Gustafson

A parallel program was measured on $p \in \{1, 2, 4, 8, 16\}$ processors. The fastest sequential program runs in $T_{\text{seq}} = 100$ seconds. The parallel runtimes are:

p	1	2	4	8	16
$T_{\text{par}}(p)$ [s]	100	55	32	20	14

1. Compute the speedup $S(p)$ and the efficiency $E(p)$ for each p. Round to two decimals.
2. Compute the total parallel overhead $O(p) = pT_{\text{par}}(p) - T_{\text{seq}}$ for each p. Briefly interpret how the overhead grows with p.
3. Use Amdahl's Law to estimate the sequential fraction f from the measurement at $p = 16$. What is the theoretical upper bound on speedup as $p \to \infty$ for this f?
4. Now assume the problem is scaled with p (*weak scaling* setting). Apply Gustafson's Law and explain in 2–3 sentences why Gustafson's prediction is more optimistic than Amdahl's, and under which assumption this holds.

Task 3 · `exercise_three.c`

Consider the following program `exercise_three.c`:

```c
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>

int main(int argc, char** argv) {
  #pragma omp parallel
  printf("I am parallel\n");
  printf("I am not parallel\n");
}
```

1. Review the program and discuss with your seatmate what the program does and why.
2. Compile the program:

```
gcc -fopenmp -o exercise_three exercise_three.c
```

3. Run the program with different numbers of threads:

```
OMP_NUM_THREADS=2 ./exercise_three
# or:
export OMP_NUM_THREADS=2 && ./exercise_three
```

4. Modify the program so that both print statements are executed by all threads.
5. Additionally, for each print statement, output the corresponding thread ID and the total number of threads. Outside the parallel region, print the number of threads again. What do you observe?

Task 4 · Degree of Parallelism—Additional, not required

Consider the following dependency DAG. Each node represents one computation operation with unit cost (1 time unit). An edge $u \to v$ means v must execute after u.

$A \to B,$	$A \to C,$	$A \to D$
$B \to E,$	$C \to E$	
$D \to F$		
$E \to G,$	$F \to G$	
$G \to H$		

So the graph contains 8 nodes $\{A, B, C, D, E, F, G, H\}$ with the edges listed above.

1. Compute the total sequential work T_{seq} (sum of node costs).
2. Compute the critical-path length T_∞ (longest path through the DAG, in time units). Name the nodes that lie on the critical path.
3. Compute the degree of parallelism $P = T_{\text{seq}}/T_\infty$. What does this number mean for the maximum achievable speedup?
4. *Bonus:* Construct a valid schedule on $p = 2$ processors. State the resulting parallel time $T_{\text{par}}(2)$ and the speedup $S(2)$. Is the schedule optimal?